

Cours — Créer une interface avec EJS et gérer des formulaires avec Express

Formation Express / EJS - Niveau 1

Objectif général

Dans ce cours, nous allons apprendre à créer une interface web avec Express et EJS.

Jusqu'ici, nous avons surtout utilisé Express pour répondre avec du texte, du HTML simple ou des données.

Maintenant, nous allons apprendre à afficher de vraies pages dynamiques.

À la fin du cours, l'application permettra de :

- afficher une page d'accueil ;
- afficher une liste de produits ;
- afficher le détail d'un produit ;
- organiser le code avec routes et contrôleurs ;
- utiliser EJS comme moteur de vues ;
- envoyer des données du serveur vers une vue ;
- utiliser des boucles et conditions dans EJS ;
- réutiliser des morceaux d'interface avec les partials ;
- créer un formulaire de recherche en GET ;
- créer un formulaire d'ajout en POST ;

- récupérer les données avec req.query et req.body ;

- afficher un message utilisateur après une action.

Les données resteront dans un tableau JavaScript pour le moment.

MongoDB viendra plus tard.

1. Création du projet

Objectif de cette partie

Créer une base Express simple pour commencer calmement.

Nous allons d'abord afficher une page simple avec Express, puis nous remplacerons progressivement `res.send()` par `res.render()`.

Commandes à exécuter

Dans un dossier de travail :

```
mkdir gestion-produits-ejs
cd gestion-produits-ejs
npm init -y
npm install express ejs
npm install --save-dev nodemon
```

Modifier package.json

Ouvrir le fichier package.json et l'adapter comme ceci :

```
{
  "name": "gestion-produits-ejs",
  "version": "1.0.0",
  "type": "module",
```

```
"scripts": {
  "start": "node app.js",
  "dev": "nodemon app.js"
},
"dependencies": {
  "ejs": "^3.1.10",
  "express": "^5.1.0"
},
"devDependencies": {
  "nodemon": "^3.1.10"
}
}
```

Explication

```
"type": "module"
```

permet d'utiliser la syntaxe moderne :

```
import express from "express";
```

au lieu de :

```
const express = require("express");
```

Le script :

```
"dev": "nodemon app.js"
```

permet de lancer le serveur avec redémarrage automatique quand on modifie le code.

2. Première application Express simple

Objectif

Avant d'utiliser EJS, on vérifie que le serveur Express fonctionne.

Créer le fichier :

```
app.js
```

Code :

```
import express from "express";

const app = express();
const PORT = 3000;

app.get("/", (req, res) => {
  res.send("Bienvenue dans l'application produits");
});

app.listen(PORT, () => {
```

```
  console.log(`Serveur lancé sur http://localhost:${PORT}`);
```

```
});
```

Lancer le serveur :

```
npm run dev
```

Puis ouvrir :

```
http://localhost:3000
```

Explication

Cette route :

```
app.get("/", (req, res) => {  
  res.send("Bienvenue dans l'application produits");  
});
```

signifie :

Quand le navigateur demande la page d'accueil,

Express répond avec un simple texte.

Pour l'instant, nous utilisons :

```
res.send()
```

`res.send()` envoie directement une réponse au navigateur.

C'est pratique pour commencer, mais limité pour construire de vraies pages HTML.

3. Pourquoi utiliser EJS ?

Problème avec `res.send()`

Si on veut afficher une vraie page HTML, on pourrait écrire :

```
res.send("<h1>Bienvenue</h1><p>Liste des produits</p>");
```

Mais ce n'est pas pratique.

Imaginez une vraie page avec :

- un menu ;
- un titre ;
- une liste de produits ;
- un formulaire ;
- un footer ;
- des conditions ;
- des boucles.

Le fichier JavaScript deviendrait rapidement difficile à lire.

Solution : utiliser un moteur de vues

EJS est un moteur de vues.

Un moteur de vues permet de créer des pages HTML dynamiques côté serveur.

Avec EJS, on sépare :

le traitement côté serveur

et

l'affichage HTML.

Le principe est simple :

Express reçoit la demande.

Le contrôleur prépare les données.

EJS utilise ces données pour fabriquer du HTML.

Le navigateur reçoit du HTML final.

Important :

Le navigateur ne reçoit pas du code EJS.

Le navigateur reçoit uniquement du HTML.

4. Première vue EJS très simple

Objectif

Remplacer progressivement `res.send()` par `res.render()`.

Créer le dossier views

Créer un dossier :

views

Dans ce dossier, créer le fichier :

home.ejs

Contenu de views/home.ejs :

```
<!DOCTYPE html>
<html lang="fr">
<head>

<meta charset="UTF-8">

<title>Accueil</title>

</head>
<body>
  <h1>Bienvenue</h1>
  <p>Première page affichée avec EJS.</p>
</body>
</html>
```

Modifier app.js

Remplacer le contenu de app.js par :

```
import express from "express";

const app = express();
const PORT = 3000;

// Configuration de EJS

app.set("view engine", "ejs");

app.get("/", (req, res) => {
  res.render("home");
});

app.listen(PORT, () => {
  console.log(`Serveur lancé sur http://localhost:${PORT}`);
});
```

Tester

Dans le navigateur :

<http://localhost:3000>

On obtient maintenant une page HTML venant du fichier :

views/home.ejs

Explication

Cette ligne :

```
app.set("view engine", "ejs");
```

dit à Express :

Quand j'utilise `res.render()`,

je veux utiliser EJS comme moteur de vues.

Cette ligne :

```
res.render("home");
```

dit à Express :

Va chercher le fichier `views/home.ejs`

et utilise-le pour générer la réponse HTML.

On vient donc de passer de :

```
res.send("Bienvenue");
```

à :

```
res.render("home");
```

5. Envoyer une donnée à une vue

Objectif

Comprendre comment le serveur peut envoyer des données vers une page EJS.

Pour le moment, notre vue contient du texte fixe.

Nous allons maintenant envoyer le titre depuis Express.

Modifier la route d'accueil

Dans `app.js` :

```
app.get("/", (req, res) => {  
  res.render("home", {  
    title: "Accueil"  
  });  
});
```

Modifier `views/home.ejs`

```
<!DOCTYPE html>  
<html lang="fr">  
<head>
```

```
<meta charset="UTF-8">
```

```
<title><%= title %></title>  
</head>  
<body>  
<h1><%= title %></h1>  
<p>Première page affichée avec EJS.</p>  
</body>  
</html>
```

Explication

Dans :

```
res.render("home", {  
  title: "Accueil"  
});
```

le deuxième argument est un objet.

Cet objet contient les données envoyées à la vue.

Ici, on envoie :

```
title: "Accueil"
```

Dans EJS, on affiche cette valeur avec :

```
<%= title %>
```

À retenir :

```
<%= %>
```

 sert à afficher une valeur dans le HTML.

6. Ajouter une page produits simple

Objectif

Créer une deuxième page pour commencer le projet produits.

À ce stade, on ne va pas encore créer de contrôleurs ni de routes séparées.

On reste simple pour comprendre la logique.

Modifier app.js

Ajouter une route /products :

```
import express from "express";  
  
const app = express();  
const PORT = 3000;  
  
app.set("view engine", "ejs");  
  
app.get("/", (req, res) => {  
  res.render("home", {  
    title: "Accueil"  
  });  
});  
  
app.get("/products", (req, res) => {  
  res.render("products", {  
    title: "Liste des produits"  
  });  
});  
  
app.listen(PORT, () => {
```

```
  console.log(`Serveur lancé sur http://localhost:${PORT}`);
```

```
});
```

Créer views/products.ejs

```
<!DOCTYPE html>  
<html lang="fr">
```

```

<head>
<meta charset="UTF-8">
<title><%= title %></title>
</head>
<body>
<h1><%= title %></h1>

<p>Ici, nous afficherons bientôt les produits.</p>

<a href="/">Retour à l'accueil</a>
</body>
</html>

```

Tester

Dans le navigateur :

`http://localhost:3000/products`

Explication

Nous avons maintenant deux routes :

```
app.get("/", ...)
```

pour la page d'accueil.

Et :

```
app.get("/products", ...)
```

pour la page produits.

Chaque route affiche une vue différente :

`/` → `views/home.ejs`

```
/products → views/products.ejs
```

7. Envoyer un tableau de produits à la vue

Objectif

Afficher des données dynamiques dans une page EJS.

Nous allons créer un tableau de produits dans `app.js`.

Plus tard, ce tableau sera remplacé par MongoDB.

Modifier `app.js`

```

import express from "express";

const app = express();
const PORT = 3000;

app.set("view engine", "ejs");

const products = [
  { id: 1, name: "Clavier mécanique", price: 8500 },
  { id: 2, name: "Souris sans fil", price: 3200 },
  { id: 3, name: "Écran 24 pouces", price: 28000 }
];

app.get("/", (req, res) => {
  res.render("home", {

```

```

    title: "Accueil"
  });
});

app.get("/products", (req, res) => {
  res.render("products", {
    title: "Liste des produits",

```

products: products

```

  });
});

app.listen(PORT, () => {
  console.log(`Serveur lancé sur http://localhost:${PORT}`);
});

```

Modifier views/products.ejs

```

<!DOCTYPE html>
<html lang="fr">
<head>

<meta charset="UTF-8">

  <title><%= title %></title>
</head>
<body>
  <h1><%= title %></h1>

  <ul>
    <% products.forEach((product) => { %>
      <li>
        <%= product.name %> - <%= product.price %> DA
      </li>
    <% }) %>
  </ul>

  <a href="/">Retour à l'accueil</a>
</body>
</html>

```

Explication

Dans le contrôleur temporaire de la route, on envoie :

```

res.render("products", {
  title: "Liste des produits",

```

products: products

```

});

```

La vue reçoit donc deux données :

```

title
products

```

products est un tableau.

Dans EJS, on peut le parcourir avec :

```

<% products.forEach((product) => { %>

```

Puis afficher chaque produit avec :

```

<%= product.name %>
<%= product.price %>
Différence entre <% %> et <%= %>

```


Dans EJS :

```
<% %>
```

sert à exécuter du JavaScript sans afficher directement le résultat.

Exemple :

```
<% products.forEach((product) => { %>
```

On démarre une boucle.

Mais on n'affiche rien directement.

```
<%= %>
```

sert à afficher une valeur dans le HTML.

Exemple :

```
<%= product.name %>
```

affiche le nom du produit.

8. Ajouter une condition dans la vue

Objectif

Afficher un message si la liste est vide.

Cela permet d'introduire les conditions dans EJS.

Modifier views/products.ejs

```
<!DOCTYPE html>  
<html lang="fr">  
<head>
```

```
<meta charset="UTF-8">
```

```
<title><%= title %></title>  
</head>  
<body>  
<h1><%= title %></h1>  
  
<% if (products.length === 0) { %>  
<p>Aucun produit disponible.</p>  
<% } else { %>  
<ul>  
<% products.forEach((product) => { %>  
<li>  
<%= product.name %> - <%= product.price %> DA  
</li>  
<% }) %>  
</ul>  
<% } %>  
  
<a href="/">Retour à l'accueil</a>  
</body>  
</html>
```

Explication

Cette condition :

```
<% if (products.length === 0) { %>
```

vérifie si le tableau est vide.

Si le tableau est vide, on affiche :

```
<p>Aucun produit disponible.</p>
```

Sinon, on affiche la liste.

Important :

La vue peut contenir une logique d'affichage simple.

Mais elle ne doit pas contenir toute la logique métier de l'application.

9. Ajouter une page détail produit

Objectif

Créer un lien vers le détail d'un produit.

Cela permet de revoir les paramètres de route avec Express.

Modifier la liste des produits

Dans `views/products.ejs`, modifier chaque produit :

```
<li>
  <%= product.name %> - <%= product.price %> DA
  <a href="/products/<%= product.id %>">Voir le détail</a>
</li>
```

La vue complète devient :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title><%= title %></title>
</head>
<body>
  <h1><%= title %></h1>

  <% if (products.length === 0) { %>
    <p>Aucun produit disponible.</p>
  <% } else { %>
    <ul>
      <% products.forEach((product) => { %>
        <li>
          <%= product.name %> - <%= product.price %> DA
          <a href="/products/<%= product.id %>">Voir le détail</a>
        </li>
      <% } %>
    </ul>
  <% } %>

  <a href="/">Retour à l'accueil</a>
</body>
</html>
```

Ajouter la route détail dans `app.js`

Ajouter cette route après la route `/products` :

```
app.get("/products/:id", (req, res) => {
  const productId = Number(req.params.id);

  const product = products.find((product) => {
    return product.id === productId;
```

```
});  
  
res.render("product-details", {  
  title: "Détail du produit",
```

product: product

```
});  
});
```

Créer views/product-details.ejs

```
<!DOCTYPE html>  
<html lang="fr">  
<head>
```

<meta charset="UTF-8">

```
  <title><%= title %></title>  
</head>  
<body>  
  <h1><%= product.name %></h1>  
  
  <p>Prix : <%= product.price %> DA</p>  
  
  <a href="/products">Retour à la liste</a>  
</body>  
</html>
```

Explication

La route :

```
app.get("/products/:id", ...)
```

permet de recevoir des URL comme :

```
/products/1  
/products/2  
/products/3
```

Dans Express :

```
req.params.id
```

contient la valeur envoyée dans l'URL.

Exemple :

```
/products/2
```

donne :

```
req.params.id
```

avec la valeur :

"2"

Attention : c'est une chaîne de caractères.

On utilise donc :

```
Number(req.params.id)
```

pour la transformer en nombre.

10. Gérer le cas produit introuvable

Objectif

Éviter une erreur si l'utilisateur demande un produit qui n'existe pas.

Par exemple :

```
http://localhost:3000/products/999
```

Modifier la route détail

Dans app.js :

```
app.get("/products/:id", (req, res) => {  
  const productId = Number(req.params.id);  
  
  const product = products.find((product) => {  
    return product.id === productId;  
  });  
  
  if (!product) {  
    return res.status(404).send("Produit introuvable");  
  }  
  
  res.render("product-details", {  
    title: "Détail du produit",  

```

product: product

```
  });  
});
```

Explication

```
if (!product) {  
  return res.status(404).send("Produit introuvable");  
}
```

signifie :

Si aucun produit n'a été trouvé,

on renvoie une réponse 404.

Pour le moment, on garde un simple `res.send()`.

Plus tard, on créera une vraie page 404 avec EJS.

11. Premier bilan

À ce stade, l'application contient :

- une page d'accueil ;
- une page liste des produits ;
- une page détail produit ;
- des données envoyées à EJS ;
- une boucle EJS ;
- une condition EJS ;
- un paramètre de route.

Mais tout est encore dans `app.js`.

Le fichier commence à grossir.

Nous allons maintenant organiser le code.

12. Séparer les données

Objectif

Sortir le tableau de produits de app.js.

Créer un dossier :

data

Puis créer :

```
data/products.js
```

Code de data/products.js

```
export const products = [
  {
    id: 1,
    name: "Clavier mécanique",
    price: 8500,
    category: "Accessoires",
    description: "Un clavier confortable pour écrire du code."
  },
  {
    id: 2,
    name: "Souris sans fil",
    price: 3200,
    category: "Accessoires",
    description: "Une souris simple et pratique pour le travail quotidien."
  },
  {
    id: 3,
    name: "Écran 24 pouces",
    price: 28000,
    category: "Écrans",
    description: "Un écran adapté au développement web et à la bureautique."
  }
];
```

Modifier app.js

En haut de app.js, importer les produits :

```
import { products } from "./data/products.js";
```

Et supprimer l'ancien tableau de produits dans app.js.

Explication

Avant, app.js contenait :

- la configuration Express ;
- les routes ;
- les données ;
- la logique de recherche ;
- l'affichage.

Ce n'est pas idéal.

Maintenant, les données sont dans un fichier séparé :

```
data/products.js
```

C'est plus clair.

13. Séparer les contrôleurs

Objectif

Créer des fonctions dédiées au traitement des routes.

Un contrôleur est un fichier qui contient les fonctions appelées par les routes.

Il reçoit la requête, prépare les données, puis renvoie une réponse.

Créer le dossier controllers

Créer :

controllers

Puis :

```
controllers/productController.js
```

Code de controllers/productController.js

```
import { products } from "../data/products.js";
```

```
export function getProducts(req, res) {  
  res.render("products", {  
    title: "Liste des produits",
```

products: products

```
  });  
}  
  
export function getProductDetails(req, res) {  
  const productId = Number(req.params.id);  
  
  const product = products.find((product) => {  
    return product.id === productId;  
  });  
  
  if (!product) {  
    return res.status(404).send("Produit introuvable");  
  }  
  
  res.render("product-details", {  
    title: "Détail du produit",
```

product: product

```
  });  
}
```

Explication

Avant, la logique était directement dans app.js.

Maintenant, on a deux fonctions :

getProducts

pour afficher la liste.

Et :

getProductDetails

pour afficher le détail.

Le contrôleur prépare les données et appelle la vue.

14. Séparer les routes produits

Objectif

Créer un fichier dédié aux routes des produits.

Créer le dossier :

routes

Puis :

```
routes/productRoutes.js
```

Code de routes/productRoutes.js

```
import express from "express";

import {
  getProducts,
  getProductDetails
} from "../controllers/productController.js";

const router = express.Router();

router.get("/", getProducts);
router.get("/:id", getProductDetails);

export default router;
```

Modifier app.js

Le fichier app.js devient plus simple :

```
import express from "express";
import productRoutes from "../routes/productRoutes.js";

const app = express();
const PORT = 3000;

app.set("view engine", "ejs");

app.get("/", (req, res) => {
  res.render("home", {
    title: "Accueil"
  });
});

app.use("/products", productRoutes);

app.listen(PORT, () => {
  console.log(`Serveur lancé sur http://localhost:${PORT}`);
});
```

Explication

Cette ligne :

```
app.use("/products", productRoutes);
```

signifie :

Toutes les routes qui commencent par /products

seront gérées par productRoutes.

Dans productRoutes.js, cette route :

```
router.get("/", getProducts);
```

correspond donc à :

```
/products
```

Et cette route :

```
router.get("/:id", getProductDetails);
```

correspond à :

```
/products/1  
/products/2  
/products/3
```

15. Séparer aussi la page d'accueil

Objectif

Avoir une structure cohérente.

Créer :

```
controllers/pageController.js
```

Code de controllers/pageController.js

```
import { products } from "../data/products.js";  
  
export function getHomePage(req, res) {  
  res.render("home", {  
    title: "Accueil",  
    productCount: products.length  
  });  
}
```

productCount: products.length

```
});  
}
```

Créer routes/pageRoutes.js

```
import express from "express";  
import { getHomePage } from "../controllers/pageController.js";  
  
const router = express.Router();  
  
router.get("/", getHomePage);  
  
export default router;
```

Modifier app.js

```
import express from "express";  
  
import pageRoutes from "./routes/pageRoutes.js";  
import productRoutes from "./routes/productRoutes.js";  
  
const app = express();  
const PORT = 3000;  
  
app.set("view engine", "ejs");  
  
app.use("/", pageRoutes);  
app.use("/products", productRoutes);
```



```
app.listen(PORT, () => {  
  console.log(`Serveur lancé sur http://localhost:${PORT}`);  
});
```

Modifier views/home.ejs

```
<!DOCTYPE html>  
<html lang="fr">  
<head>  
  <meta charset="UTF-8">  
  <title><%= title %></title>  
</head>  
<body>  
  <h1><%= title %></h1>  
  
  <p>Bienvenue dans l'application de gestion des produits.</p>  
  
<p>
```

Nombre de produits disponibles :

```
<strong><%= productCount %></strong>  
</p>  
  
<a href="/products">Voir les produits</a>  
</body>  
</html>
```

16. Bilan de structuration

La structure actuelle devient :

```
gestion-produits-ejs/  
├── app.js  
├── data/  
│   └── products.js  
├── controllers/  
│   ├── pageController.js  
│   └── productController.js  
├── routes/  
│   ├── pageRoutes.js  
│   └── productRoutes.js  
└── views/  
    ├── home.ejs  
    ├── products.ejs  
    └── product-details.ejs
```

Rôle de chaque dossier

app.js

configure l'application Express.

routes/

contient les URL disponibles.

controllers/

contient les fonctions qui traitent les requêtes.

data/

contient les données temporaires.

views/

contient les pages EJS.

17. Ajouter des partials

Objectif

Éviter de répéter le même HTML dans toutes les pages.

Pour le moment, chaque vue contient :

```
<!DOCTYPE html>
<html>
<head>
```

...

```
</head>
<body>
```

...

```
</body>
</html>
```

On répète beaucoup de code.

Nous allons créer des partials.

Un partial est un morceau de page réutilisable.

Exemples :

- header ;
- menu ;
- footer.

Créer le dossier

Dans views, créer :

partials

Créer views/partials/header.ejs

```
<!DOCTYPE html>
<html lang="fr">
<head>
```

```
<meta charset="UTF-8">
```

```
<title><%= title %></title>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
<h1>Gestion des produits</h1>
```

```
</header>
```

Créer views/partials/navbar.ejs

```
<nav>
```

```
<a href="/">Accueil</a>
```

```
<a href="/products">Produits</a>
```

```
</nav>
```

```
<hr>
```

Créer views/partials/footer.ejs

```
<hr>
```

```
<footer>
```

```
  <p>Formation Express / EJS — Niveau 1</p>
</footer>
</body>
</html>
```

Modifier views/home.ejs

```
<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
  <h2><%= title %></h2>

  <p>Bienvenue dans l'application de gestion des produits.</p>

  <p>
```

Nombre de produits disponibles :

```
  <strong><%= productCount %></strong>
</p>

  <a href="/products">Voir les produits</a>
</main>

<%- include("partials/footer") %>
```

Modifier views/products.ejs

```
<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
  <h2><%= title %></h2>

  <% if (products.length === 0) { %>
    <p>Aucun produit disponible.</p>
  <% } else { %>
    <ul>
      <% products.forEach((product) => { %>
        <li>
          <%= product.name %> - <%= product.price %> DA
          <a href="/products/<%= product.id %>">Voir le détail</a>
        </li>
      <% }) %>
    </ul>
  <% } %>
</main>

<%- include("partials/footer") %>
```

Modifier views/product-details.ejs

```
<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
```

```

<h2><%= product.name %></h2>

<p>Prix : <%= product.price %> DA</p>

<p>Catégorie : <%= product.category %></p>

<p><%= product.description %></p>

<a href="/products">Retour à la liste</a>
</main>

<%- include("partials/footer") %>

```

Explication

Cette ligne :

```
<%- include("partials/header") %>
```

inclut le fichier :

```
views/partials/header.ejs
```

On utilise :

```
<%= %>
```

parce qu'on veut insérer du HTML.

À retenir :

```

<%= %> affiche une valeur.
<% %> exécute du JavaScript.
<%- %> affiche du HTML non échappé, utile pour les includes.

```

18. Ajouter un peu de CSS

Objectif

Rendre l'interface plus lisible sans perdre trop de temps.

Modifier app.js

Pour utiliser un dossier public, on ajoute :

```
app.use(express.static("public"));
```

Le fichier app.js devient :

```

import express from "express";

import pageRoutes from "./routes/pageRoutes.js";
import productRoutes from "./routes/productRoutes.js";

const app = express();
const PORT = 3000;

app.set("view engine", "ejs");

app.use(express.static("public"));

app.use("/", pageRoutes);
app.use("/products", productRoutes);

app.listen(PORT, () => {

```

```
console.log(`Serveur lancé sur http://localhost:${PORT}`);
```

```
});
```

Modifier views/partials/header.ejs

Ajouter le lien CSS :

```
<!DOCTYPE html>
<html lang="fr">
<head>

<meta charset="UTF-8">

<title><%= title %></title>
<link rel="stylesheet" href="/css/style.css">
</head>
<body>
<header class="site-header">
  <h1>Gestion des produits</h1>
</header>
```

Créer le fichier CSS

Créer :

```
public/css/style.css
```

Code simple :

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f5f5f5;
  color: #222;
}

.site-header {
  background: #1f2937;
  color: white;
  padding: 20px;
}

nav {
  background: white;
  padding: 15px 20px;
}

nav a {
  margin-right: 15px;
  text-decoration: none;
  color: #2563eb;
}

main {
  padding: 20px;
  max-width: 900px;
  margin: auto;
  background: white;
  min-height: 300px;
}

ul {
  padding-left: 20px;
}

li {
```

```
margin-bottom: 10px;
}

a {
  color: #2563eb;
}

footer {
  text-align: center;
  padding: 15px;
  color: #666;
}
```

19. Introduction aux formulaires

Objectif

Comprendre comment un utilisateur peut envoyer des informations au serveur.

Jusqu'ici, le navigateur demandait seulement des pages.

Maintenant, l'utilisateur va pouvoir envoyer des données.

Il existe deux méthodes importantes :

GET

POST

GET

GET est utilisé pour demander une ressource ou faire une recherche.

Exemple :

```
/products?search=clavier
```

On peut lire ces informations avec :

```
req.query
```

POST

POST est utilisé pour envoyer des données qui créent ou modifient quelque chose.

Exemple :

Ajouter un produit.

On peut lire ces informations avec :

```
req.body
```

Mais pour cela, il faut activer :

express.urlencoded()

20. Ajouter un formulaire de recherche avec GET

Objectif

Créer un premier formulaire simple.

La recherche est une bonne introduction, car elle ne modifie pas les données.

Modifier views/products.ejs

Ajouter ce formulaire sous le titre :

```
<form action="/products" method="GET">
  <input type="text" name="search" placeholder="Rechercher un produit">
  <button type="submit">Rechercher</button>
</form>
```

La vue devient :

```
<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
  <h2><%= title %></h2>

  <form action="/products" method="GET">
    <input type="text" name="search" placeholder="Rechercher un produit">
    <button type="submit">Rechercher</button>
  </form>

  <% if (products.length === 0) { %>
    <p>Aucun produit disponible.</p>
  <% } else { %>
    <ul>
      <% products.forEach((product) => { %>
        <li>
          <%= product.name %> - <%= product.price %> DA
          <a href="/products/<%= product.id %>">Voir le détail</a>
        </li>
      <% } %>
    </ul>
  <% } %>
</main>

<%- include("partials/footer") %>
```

Tester

Dans le champ de recherche, écrire :

clavier

Puis cliquer sur Rechercher.

L'URL devient :

```
/products?search=clavier
```

Pour le moment, rien n'est filtré.

C'est normal.

Nous avons seulement créé le formulaire.

```
21. Lire la recherche avec req.query
```

Objectif

Récupérer la valeur envoyée par le formulaire GET.

Modifier controllers/productController.js

Modifier la fonction getProducts :

```
export function getProducts(req, res) {
```

```
const search = req.query.search || "";

let filteredProducts = products;

if (search !== "") {
  filteredProducts = products.filter((product) => {
    return product.name.toLowerCase().includes(search.toLowerCase());
  });
}

res.render("products", {
  title: "Liste des produits",
  products: filteredProducts,

```

search: search

```
});
}
```

Modifier views/products.ejs

Dans le formulaire, modifier l'input :

```
<input
type="text"
name="search"
placeholder="Rechercher un produit"
value="<%= search %>"
>
```

Puis ajouter un message si une recherche est active :

```
<% if (search) { %>
<p>
```

Recherche actuelle :

```
<strong><%= search %></strong>
</p>

<a href="/products">Réinitialiser</a>
<% } %>
```

Vue complète :

```
<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
<h2><%= title %></h2>

<form action="/products" method="GET">
<input
```

```
type="text"
name="search"
placeholder="Rechercher un produit"
value="<%= search %>"
>
```

```
<button type="submit">Rechercher</button>
</form>
```



```
<% if (search) { %>
<p>
```

Recherche actuelle :

```
<strong><%= search %></strong>
</p>

<a href="/products">Réinitialiser</a>
<% } %>

<% if (products.length === 0) { %>
<p>Aucun produit disponible.</p>
<% } else { %>
<ul>
  <% products.forEach((product) => { %>
    <li>
      <%= product.name %> - <%= product.price %> DA
      <a href="/products/<%= product.id %>">Voir le détail</a>
    </li>
  <% } } %>
</ul>
<% } %>
</main>

<%- include("partials/footer") %>
```

Explication

Quand le formulaire est envoyé en GET :

```
<form action="/products" method="GET">
```

le navigateur ajoute les données dans l'URL.

Le champ :

```
<input name="search">
```

donne une URL comme :

```
/products?search=clavier
```

Côté serveur, Express permet de lire cette valeur avec :

```
req.query.search
```

Donc :

```
const search = req.query.search || "";
```

signifie :

Je récupère la recherche envoyée dans l'URL.

S'il n'y a pas de recherche, j'utilise une chaîne vide.

22. Formulaire d'ajout : afficher le formulaire

Objectif

Créer une page pour ajouter un produit.

Important : on va d'abord afficher le formulaire.

On ne va pas encore le traiter.

C'est une distinction essentielle :

GET /products/new

sert à afficher le formulaire.

POST /products

servira à traiter le formulaire.

Modifier routes/productRoutes.js

Attention à l'ordre.

Ajouter la route /new avant /:id :

```
import express from "express";

import {
  getProducts,
  getProductDetails,
  showCreateProductForm
} from "../controllers/productController.js";

const router = express.Router();

router.get("/", getProducts);
router.get("/new", showCreateProductForm);
router.get("/:id", getProductDetails);

export default router;
```

Pourquoi l'ordre est important ?

Si on écrit :

```
router.get("/:id", getProductDetails);
router.get("/new", showCreateProductForm);
```

Express peut comprendre :

```
/products/new
```

comme :

```
/products/:id
```

avec :

id = "new"

Donc on place toujours les routes précises avant les routes dynamiques.

Modifier controllers/productController.js

Ajouter cette fonction :

```
export function showCreateProductForm(req, res) {
  res.render("product-form", {
    title: "Ajouter un produit"
  });
}
```

Créer views/product-form.ejs

```
<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
  <h2><%= title %></h2>
```

```

<p>Remplissez le formulaire pour ajouter un nouveau produit.</p>

<form action="/products" method="POST">
  <div>
    <label for="name">Nom du produit</label>
    <input type="text" id="name" name="name">
  </div>

  <div>
    <label for="price">Prix</label>
    <input type="number" id="price" name="price">
  </div>

  <div>
    <label for="category">Catégorie</label>
    <input type="text" id="category" name="category">
  </div>

  <div>
    <label for="description">Description</label>
    <textarea id="description" name="description"></textarea>
  </div>

  <button type="submit">Ajouter</button>
</form>
</main>

<%- include("partials/footer") %>

```

Modifier la navigation

Dans views/partials/navbar.ejs :

```

<nav>
  <a href="/">Accueil</a>
  <a href="/products">Produits</a>
  <a href="/products/new">Ajouter un produit</a>
</nav>

<hr>

```

Tester

Dans le navigateur :

<http://localhost:3000/products/new>

On doit voir le formulaire.

Pour le moment, si on clique sur “Ajouter”, cela ne marche pas encore.

C’est normal.

Nous avons seulement créé la page du formulaire.

23. Comprendre les champs d’un formulaire

Dans un formulaire HTML, l’attribut le plus important côté serveur est :

name

Exemple :

```
<input type="text" name="name">
```

Quand le formulaire sera envoyé, Express pourra lire :

```
req.body.name
```

Autre exemple :

```
<input type="number" name="price">
```

donnera :

```
req.body.price
```

Donc le lien est direct :

```
name="name" → req.body.name
```

```
name="price" → req.body.price
```

24. Activer la lecture des formulaires POST

Objectif

Permettre à Express de lire les données envoyées par un formulaire HTML.

Dans app.js, ajouter cette ligne avant les routes :

```
app.use(express.urlencoded({ extended: true }));
```

Le fichier app.js devient :

```
import express from "express";

import pageRoutes from "./routes/pageRoutes.js";
import productRoutes from "./routes/productRoutes.js";

const app = express();
const PORT = 3000;

app.set("view engine", "ejs");

app.use(express.static("public"));
```

// Permet de lire les données envoyées par un formulaire HTML

```
app.use(express.urlencoded({ extended: true }));

app.use("/", pageRoutes);
app.use("/products", productRoutes);

app.listen(PORT, () => {
  console.log(`Serveur lancé sur http://localhost:${PORT}`);
});
```

Explication

Sans cette ligne :

```
app.use(express.urlencoded({ extended: true }));
```

Express ne peut pas lire correctement les données envoyées par un formulaire HTML classique.

Grâce à cette ligne, on pourra utiliser :

```
req.body
```

dans le contrôleur.

25. Traiter le formulaire avec POST

Objectif

Créer une route qui reçoit les données du formulaire.

Modifier routes/productRoutes.js

Ajouter createProduct dans l'import :

```
import {  
  getProducts,  
  getProductDetails,  
  showCreateProductForm,  
  createProduct  
} from "../controllers/productController.js";
```

Puis ajouter la route POST :

```
router.post("/", createProduct);
```

Le fichier complet devient :

```
import express from "express";  
  
import {  
  getProducts,  
  getProductDetails,  
  showCreateProductForm,  
  createProduct  
} from "../controllers/productController.js";  
  
const router = express.Router();  
  
router.get("/", getProducts);  
router.get("/new", showCreateProductForm);  
router.post("/", createProduct);  
router.get("/:id", getProductDetails);  
  
export default router;
```

Modifier controllers/productController.js

Ajouter la fonction :

```
export function createProduct(req, res) {  
  const newProduct = {  
    id: products.length + 1,  
    name: req.body.name,  
    price: Number(req.body.price),  
    category: req.body.category,  
    description: req.body.description  
  };  
  
  products.push(newProduct);  
  
  res.redirect("/products");  
}
```

Tester

Aller sur :

```
http://localhost:3000/products/new
```

Remplir :

Nom : Casque audio

Prix : 6500

Catégorie : Audio

Description : Casque confortable pour travailler.

Cliquer sur Ajouter.

On doit être redirigé vers :

```
/products
```

Le nouveau produit doit apparaître dans la liste.

Explication

Le formulaire envoie les données ici :

```
<form action="/products" method="POST">
```

Donc il appelle :

POST /products

Dans les routes :

```
router.post("/", createProduct);
```

correspond à :

POST /products

puisque dans app.js, on a :

```
app.use("/products", productRoutes);  
Comprendre req.body
```

Dans le formulaire :

```
<input name="name">
```

donne :

```
req.body.name
```

Dans le formulaire :

```
<input name="price">
```

donne :

```
req.body.price
```

Dans le formulaire :

```
<textarea name="description"></textarea>
```

donne :

```
req.body.description
```

26. Améliorer l'identifiant du nouveau produit

Objectif

Éviter un problème si les identifiants ne suivent pas exactement la longueur du tableau.

Pour le moment, on utilise :

```
id: products.length + 1
```

C'est acceptable pour débiter.

Mais on peut faire un peu mieux.

Ajouter une fonction dans productController.js

En bas du fichier :

```
function getNextProductId() {  
  if (products.length === 0) {  
    return 1;  
  }  
  
  const ids = products.map((product) => product.id);  
  
  return Math.max(...ids) + 1;  
}
```

Puis modifier createProduct :

```
export function createProduct(req, res) {  
  const newProduct = {  
    id: getNextProductId(),  
    name: req.body.name,  
    price: Number(req.body.price),  
    category: req.body.category,  
    description: req.body.description  
  };  
  
  products.push(newProduct);  
  
  res.redirect("/products");  
}
```

Explication

Cette fonction cherche le plus grand identifiant existant, puis ajoute 1.

Exemple :

Produits existants : 1, 2, 3

Nouveau produit : 4

27. Ajouter une validation simple

Objectif

Éviter d'ajouter un produit vide.

On reste simple.

Pas de validation avancée pour l'instant.

Modifier createProduct

```
export function createProduct(req, res) {
```

```

const name = req.body.name;
const price = req.body.price;
const category = req.body.category;
const description = req.body.description;

if (!name || !price || !description) {
  return res.status(400).render("product-form", {
    title: "Ajouter un produit",

```

error: "Veuillez remplir au minimum le nom, le prix et la description.",

```

    oldInput: {
      name: name,
      price: price,
      category: category,
      description: description
    }
  });
}

const newProduct = {
  id: getNextProductId(),
  name: name,
  price: Number(price),
  category: category || "Non classé",
  description: description
};

products.push(newProduct);

res.redirect("/products");
}

```

Modifier showCreateProductForm

Comme la vue attend maintenant error et oldInput, on modifie aussi :

```

export function showCreateProductForm(req, res) {
  res.render("product-form", {
    title: "Ajouter un produit",
    error: "",
    oldInput: {
      name: "",
      price: "",
      category: "",
      description: ""
    }
  });
}

```

Modifier views/product-form.ejs

```

<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
  <h2><%= title %></h2>

  <p>Remplissez le formulaire pour ajouter un nouveau produit.</p>

  <% if (error) { %>
    <p class="error"><%= error %></p>
  <% } %>

  <form action="/products" method="POST">
    <div>

```



```
<label for="name">Nom du produit</label>
<input
```

type="text"

id="name"

name="name"

```
value="<%= oldInput.name %>"
```

>

```
</div>
```

```
<div>
```

```
<label for="price">Prix</label>
```

```
<input
```

type="number"

id="price"

name="price"

```
value="<%= oldInput.price %>"
```

>

```
</div>
```

```
<div>
```

```
<label for="category">Catégorie</label>
```

```
<input
```

type="text"

id="category"

name="category"

```
value="<%= oldInput.category %>"
```

>

```
</div>
```

```
<div>
```

```
<label for="description">Description</label>
```

```
<textarea
```

id="description"

name="description"

```
><%= oldInput.description %></textarea>
```

```
</div>
```

```
<button type="submit">Ajouter</button>
```

```
</form>
```

```
</main>
```

```
<%- include("partials/footer") %>
```

Explication

On vérifie :

```
if (!name || !price || !description)
```

Cela veut dire :

Si le nom, le prix ou la description manque,

on réaffiche le formulaire avec un message d'erreur.

On renvoie aussi :

```
oldInput
```

pour garder les valeurs déjà saisies.

Cela évite à l'utilisateur de tout retaper.

28. Ajouter un message de succès

Objectif

Afficher un retour utilisateur après l'ajout d'un produit.

Pour rester niveau 1, on va utiliser une solution simple avec query string.

Après l'ajout, au lieu de faire :

```
res.redirect("/products");
```

on va faire :

```
res.redirect("/products?message=created");
```

Modifier createProduct

```
products.push(newProduct);  
  
res.redirect("/products?message=created");
```

Modifier getProducts

```
export function getProducts(req, res) {  
  const search = req.query.search || "";  
  const message = req.query.message || "";  
  
  let filteredProducts = products;  
  
  if (search !== "") {  
    filteredProducts = products.filter((product) => {  
      return product.name.toLowerCase().includes(search.toLowerCase());  
    });  
  }  
  
  res.render("products", {  
    title: "Liste des produits",  
    products: filteredProducts,  
    search: search,  
    message: message  
  });  
}
```

Modifier views/products.ejs

Ajouter après le titre :

```
<% if (message === "created") { %>  
  <p class="success">Produit ajouté avec succès.</p>  
<% } %>
```

La vue complète devient :

```
<%- include("partials/header") %>  
<%- include("partials/navbar") %>  
  
<main>
```

```

<h2><%= title %></h2>

<% if (message === "created") { %>
  <p class="success">Produit ajouté avec succès.</p>
<% } %>

<form action="/products" method="GET">
  <input

```

type="text"

name="search"

placeholder="Rechercher un produit"

```

  value="<%= search %>"

```

>

```

  <button type="submit">Rechercher</button>
</form>

<% if (search) { %>
  <p>

```

Recherche actuelle :

```

  <strong><%= search %></strong>
</p>

  <a href="/products">Réinitialiser</a>
<% } %>

<% if (products.length === 0) { %>
  <p>Aucun produit disponible.</p>
<% } else { %>
  <ul>
    <% products.forEach((product) => { %>
      <li>
        <%= product.name %> - <%= product.price %> DA
        <a href="/products/<%= product.id %>">Voir le détail</a>
      </li>
    <% } %>
  </ul>
<% } %>
</main>

<%- include("partials/footer") %>

```

Explication

Après l'ajout, on redirige vers :

```

/products?message=created

```

Dans le contrôleur, on lit :

```

const message = req.query.message || "";

```

Puis dans la vue :

```

<% if (message === "created") { %>
  <p class="success">Produit ajouté avec succès.</p>
<% } %>

```

Pour le moment, cette solution est suffisante.

Les vrais messages flash avec sessions pourront venir plus tard.

29. Ajouter du style aux formulaires et messages

Dans `public/css/style.css`, ajouter :

```
form {
  margin: 20px 0;
}

input,
textarea,
button {
  display: block;
  margin-top: 6px;
  margin-bottom: 15px;
  padding: 8px;
  width: 100%;
  max-width: 400px;
}

button {
  width: auto;
  background: #2563eb;
  color: white;
  border: none;
  cursor: pointer;
}

button:hover {
  background: #1d4ed8;
}

.success {
  background: #dcfce7;
  color: #166534;
  padding: 10px;
  border-left: 4px solid #16a34a;
}

.error {
  background: #fee2e2;
  color: #991b1b;
  padding: 10px;
  border-left: 4px solid #dc2626;
}
```

30. Ajouter une page 404 avec EJS

Objectif

Avoir une page d'erreur propre.

Créer `views/404.ejs`

```
<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
  <h2><%= title %></h2>

  <p>La page demandée n'existe pas.</p>

  <a href="/">Retour à l'accueil</a>
</main>
```

```
<%- include("partials/footer") %>
```

Modifier le produit introuvable

Dans getProductDetails :

```
if (!product) {  
  return res.status(404).render("404", {  
    title: "Produit introuvable"  
  });  
}
```

Ajouter une 404 globale dans app.js

Avant app.listen :

```
app.use((req, res) => {  
  res.status(404).render("404", {  
    title: "Page introuvable"  
  });  
});
```

app.js complet :

```
import express from "express";  
  
import pageRoutes from "./routes/pageRoutes.js";  
import productRoutes from "./routes/productRoutes.js";  
  
const app = express();  
const PORT = 3000;  
  
app.set("view engine", "ejs");  
  
app.use(express.static("public"));  
app.use(express.urlencoded({ extended: true }));  
  
app.use("/", pageRoutes);  
app.use("/products", productRoutes);  
  
app.use((req, res) => {  
  res.status(404).render("404", {  
    title: "Page introuvable"  
  });  
});  
  
app.listen(PORT, () => {  
  console.log(`Serveur lancé sur http://localhost:${PORT}`);  
});
```

31. Version finale des fichiers principaux

```
app.js  
import express from "express";  
  
import pageRoutes from "./routes/pageRoutes.js";  
import productRoutes from "./routes/productRoutes.js";  
  
const app = express();  
const PORT = 3000;  
  
app.set("view engine", "ejs");
```

```
app.use(express.static("public"));
app.use(express.urlencoded({ extended: true }));
```

```
app.use("/", pageRoutes);
app.use("/products", productRoutes);
```

```
app.use((req, res) => {
  res.status(404).render("404", {
    title: "Page introuvable"
  });
});
```

```
app.listen(PORT, () => {
```

```
  console.log(`Serveur lancé sur http://localhost:${PORT}`);
```

```
});
```

```
data/products.js
```

```
export const products = [
```

```
  {
    id: 1,
    name: "Clavier mécanique",
    price: 8500,
    category: "Accessoires",
    description: "Un clavier confortable pour écrire du code."
  },
  {
    id: 2,
    name: "Souris sans fil",
    price: 3200,
    category: "Accessoires",
    description: "Une souris simple et pratique pour le travail quotidien."
  },
  {
    id: 3,
    name: "Écran 24 pouces",
    price: 28000,
    category: "Écrans",
    description: "Un écran adapté au développement web et à la bureautique."
  }
];
```

```
routes/pageRoutes.js
```

```
import express from "express";
```

```
import { getHomePage } from "../controllers/pageController.js";
```

```
const router = express.Router();
```

```
router.get("/", getHomePage);
```

```
export default router;
```

```
routes/productRoutes.js
```

```
import express from "express";
```

```
import {
```

```
  getProducts,
```

```
  getProductDetails,
```

```
  showCreateProductForm,
```

```
  createProduct
```

```
} from "../controllers/productController.js";
```

```
const router = express.Router();
```

```

router.get("/", getProducts);
router.get("/new", showCreateProductForm);
router.post("/", createProduct);
router.get("/:id", getProductDetails);

export default router;
controllers/pageController.js
import { products } from "../data/products.js";

export function getHomePage(req, res) {
  res.render("home", {
    title: "Accueil",

```

productCount: products.length

```

});
}
controllers/productController.js
import { products } from "../data/products.js";

export function getProducts(req, res) {
  const search = req.query.search || "";
  const message = req.query.message || "";

  let filteredProducts = products;

  if (search !== "") {
    filteredProducts = products.filter((product) => {
      return product.name.toLowerCase().includes(search.toLowerCase());
    });
  }

  res.render("products", {
    title: "Liste des produits",
    products: filteredProducts,
    search: search,
    message: message
  });
}

export function getProductDetails(req, res) {
  const productId = Number(req.params.id);

  const product = products.find((product) => {
    return product.id === productId;
  });

  if (!product) {
    return res.status(404).render("404", {
      title: "Produit introuvable"
    });
  }

  res.render("product-details", {
    title: "Détail du produit",

```

product: product

```

});
}

export function showCreateProductForm(req, res) {
  res.render("product-form", {
    title: "Ajouter un produit",
    error: "",

```

```

oldInput: {
  name: "",
  price: "",
  category: "",
  description: ""
}
});
}

export function createProduct(req, res) {
  const name = req.body.name;
  const price = req.body.price;
  const category = req.body.category;
  const description = req.body.description;

  if (!name || !price || !description) {
    return res.status(400).render("product-form", {
      title: "Ajouter un produit",

```

error: "Veuillez remplir au minimum le nom, le prix et la description.",

```

oldInput: {
  name: name,
  price: price,
  category: category,
  description: description
}
});
}

const newProduct = {
  id: getNextProductId(),
  name: name,
  price: Number(price),
  category: category || "Non classé",
  description: description
};

products.push(newProduct);

res.redirect("/products?message=created");
}

function getNextProductId() {
  if (products.length === 0) {
    return 1;
  }

  const ids = products.map((product) => product.id);

  return Math.max(...ids) + 1;
}

```

32. Structure finale

À la fin, le projet ressemble à ceci :

```

gestion-produits-ejs/
├── app.js
├── package.json
├── data/
│   └── products.js

```



```

├── controllers/
│   ├── pageController.js
│   └── productController.js
├── routes/
│   ├── pageRoutes.js
│   └── productRoutes.js
├── views/
│   ├── home.ejs
│   ├── products.ejs
│   ├── product-details.ejs
│   ├── product-form.ejs
│   ├── 404.ejs
│   └── partials/
│       ├── header.ejs
│       ├── navbar.ejs
│       └── footer.ejs
└── public/
    ├── css/
    └── style.css

```

33. Synthèse pédagogique

À retenir :

Une route reçoit une demande.

Un contrôleur prépare la réponse.

Une vue EJS affiche la page.

Le navigateur reçoit du HTML.

Pour afficher une vue :

```
res.render("nom-de-la-vue", {
```

cle: valeur

```
});
```

Pour afficher une valeur dans EJS :

```
<%= valeur %>
```

Pour exécuter du JavaScript dans EJS :

```
<% if (...) { %>
```

ou :

```
<% tableau.forEach(...) %>
```

Pour inclure un partial :

```
<%- include("partials/header") %>
```

Pour lire une recherche en GET :

```
req.query.search
```

Pour lire un formulaire en POST :

```
req.body.name
```

Mais il faut activer :

```
app.use(express.urlencoded({ extended: true }));
```

34. Exercices de consolidation

Exercice 1 — Ajouter un champ stock

Ajouter un champ stock aux produits.

Exemple :

```
stock: 15
```

Puis l'afficher :

Stock disponible : 15

dans :

- la liste des produits ;
- la page détail.

Exercice 2 — Ajouter le champ stock dans le formulaire

Dans le formulaire d'ajout, ajouter :

```
<input type="number" name="stock">
```

Puis récupérer :

```
req.body.stock
```

Et l'ajouter dans le nouveau produit.

Exercice 3 — Afficher un message si le stock est faible

Dans la vue détail :

Si le stock est inférieur à 5,

afficher : Stock faible.

Indice :

```
<% if (product.stock < 5) { %>
  <p>Stock faible</p>
<% } %>
```

Exercice 4 — Tester les cas d'erreur

Tester :

```
/products/999
```

Résultat attendu :

Page produit introuvable.

Tester aussi :

```
/une-page-inconnue
```

Résultat attendu :

Page introuvable.

35. Correction rapide de l'exercice stock

Dans data/products.js

Exemple :

```
export const products = [
  {
    id: 1,
    name: "Clavier mécanique",
    price: 8500,
    stock: 12,
    category: "Accessoires",
    description: "Un clavier confortable pour écrire du code."
  },
  {
    id: 2,
    name: "Souris sans fil",
    price: 3200,
    stock: 4,
    category: "Accessoires",
    description: "Une souris simple et pratique pour le travail quotidien."
  },
  {
    id: 3,
    name: "Écran 24 pouces",
    price: 28000,
    stock: 8,
    category: "Écrans",
    description: "Un écran adapté au développement web et à la bureautique."
  }
];
```

Dans product-form.ejs

Ajouter :

```
<div>
  <label for="stock">Stock</label>
  <input
```

type="number"

id="stock"

name="stock"

```
  value="<%= oldInput.stock %>"
```

>

```
</div>
```

Dans showCreateProductForm

Ajouter stock :

```
oldInput: {
  name: "",
  price: "",
  stock: "",
  category: "",
  description: ""
}
```

Dans createProduct

Ajouter :

```
const stock = req.body.stock;
```

Dans oldInput :

```
stock: stock,
```

Dans newProduct :

```
stock: Number(stock),
```

Dans la liste

```
<%= product.name %> - <%= product.price %> DA  
- Stock : <%= product.stock %>
```

Dans le détail

```
<p>Stock : <%= product.stock %></p>  
  
<% if (product.stock < 5) { %>  
  <p class="error">Stock faible</p>  
<% } %>
```

36. Conclusion du cours

Dans ce cours, nous sommes partis d'une route simple :

```
app.get("/", ...)
```

Puis nous avons progressivement construit une application organisée.

Nous avons vu :

- le rôle de EJS ;
- le rendu côté serveur ;

```
- res.render ;
```

- les vues ;
- les partials ;
- les données envoyées à une vue ;
- les boucles ;
- les conditions ;
- les formulaires GET ;

```
- req.query ;
```

- les formulaires POST ;

```
- req.body ;
```

- express.urlencoded ;
- les messages utilisateur ;
- l'organisation routes / contrôleurs / vues.

L'application fonctionne, mais les données sont encore temporaires.

Si on redémarre le serveur, les produits ajoutés disparaissent.

C'est normal.

La prochaine étape sera de remplacer le tableau JavaScript par une vraie base MongoDB avec Mongoose.